Rupee Quest (working title): learn money by growing a practice portfolio

Problem: Vanguard India, "Community Stewardship": a digital financial-literacy experience for Grade 11–12 students in Hyderabad.

1. Problem statement and our understanding

The brief asks for a digital experience that Grade 11–12 students from underserved Hyderabad communities actually want to use, on the theme of financial literacy. It should start with about 1,500 students and be designed to scale. It must let students **learn by doing** (make choices, see consequences, try again), and be simple, relatable, interactive and safe to explore. Four pillars: plan and budget; save and manage; understand investing; stay financially safe. Success measures: simple, relatable, persona-led, language-ready, game-based, intuitive UI, NGO-integrated. Guardrail: **educate, do not recommend investments; simulations are assumptions, not promises**; keep it privacy-conscious.

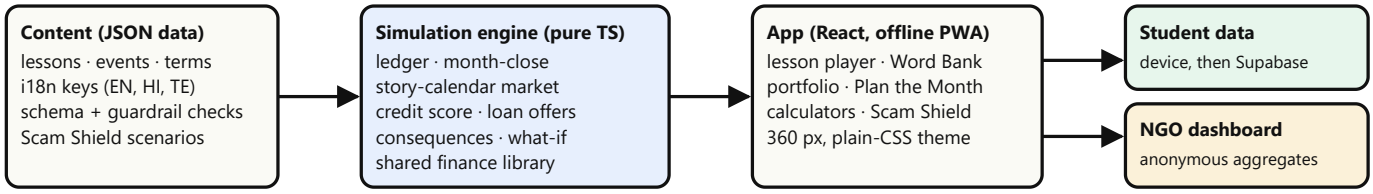
What we think is hard: (1) learners are minors on shared, low-end phones with patchy data, so it must work offline and collect no personal data; (2) "safe to make mistakes" needs consequences that feel real but cost nothing; (3) English, Hindi and Telugu, while financial terms stay recognisable; (4) NGOs need rollout and progress visibility without learner identities; (5) the no-recommendation guardrail has to be enforced, not just stated.

2. Idea / abstract

Learning **is** growing a practice portfolio. Each sub-section is one game-month: a gut-check choice, 5–8 swipe cards (plain words first, real term second), a mini-tool, and a 3-question quiz. The quiz pays practice money (₹2,000 plus ₹1,000 per correct answer), which the student places in a portfolio: wallet, savings, emergency fund, recurring deposit, fixed deposit, gold, mutual fund/SIP and shares, plus loans and "things I own" that lose value. Levels unlock these buckets in a sensible order: money basics, banking, safety net, borrowing and credit, growth, investing, taxes, fraud-proofing (9 levels, 29 sub-sections).

- **Consequences:** after each level a life event (festival sale, phone EMI offer, market crash, scam message) offers choices with good, mixed or bad outcomes, "what if" previews of the other choices, and one try-again.
- **Credit-based loans:** loan rate and maximum depend on a credit score the student builds (on-time EMIs raise it, missed ones lower it).
- **Own wants and goals:** students add their own wants any time and turn them into goals with an amount and a deadline.
- **Simplifying assumption:** needs are already covered by family, so practice money is only for wants and savings (default 60/40 split).
- **Embedded tools:** calculators with eye-opening "shockers" (SIP, EMI, inflation, FD, fees) and a Scam Shield swipe game (UPI, fake-job, loan-app and KYC scams, and where to report).
- **Reach:** English, Hindi, Telugu with the English term kept beside each translation; an NGO account with class codes, anonymous nicknames, and aggregate progress and weak spots only.

3. High-level approach and architecture



- **Content as data:** lessons, events and terms are JSON checked by schemas; every string is an i18n key, so a new language is only new files.
- **Deterministic engine:** a reducer over game state with no randomness. Month-close applies automatic payments, interest, depreciation, a fixed scripted market and the credit score; the consequence engine applies a choice and previews the alternatives. One shared finance library (SIP, EMI, FD, RD, inflation, depreciation, slab tax) feeds both the engine and the calculators.
- **App:** React + Vite + TypeScript, mobile-first, installable and offline-capable; state persists on the device in phase 1.
- **Phase 2:** Supabase (anonymous student sign-in with class code and nickname, NGO email login, row-level security) on Vercel; no personal data collected.
- **Process:** parallel lanes (engine and events; lessons and home; calculators and Scam Shield) merging to main by pull request, with CI running typecheck, tests and build.

4. Planned testing methodology

- **Formula correctness:** unit tests against hand-computed values (for example SIP of ₹10,000 a month for 15 years at 10% = ₹41.79 lakh; EMI on ₹6,000 over 6 months at 18% = ₹1,053; FD ₹6,000 for 6 months at 6.5% = ₹6,195). The foundation

already ships 79 automated tests.

- **Engine behaviour:** deterministic replay (same actions give the same state), month-close ordering, shortfall and credit-score rules, scripted market episodes, health-score band boundaries, and "what if" leaving state untouched.
- **Content checks on every pull request:** schema validation, every i18n key exists, no duplicate keys, exactly 3 quiz questions per lesson, terms defined, unverified facts hidden, and a guardrail lint (no product names, no advice wording).
- **UI checks:** route and render tests; manual runs at 360 px width, in airplane mode and by keyboard; two complete playthroughs before release.
- **Learning evidence (stated honestly):** first-attempt quiz scores per concept and a 10-question final quiz. With no pre-test baseline we report scores, not "improvement"; a small readability pilot with Grade 11–12 students is planned if time allows.
- **Privacy and safety review:** no personal data stored; scenarios use no real brands or people.

Known limits: simulations use fixed pretend market paths and assumed rates; some regulatory figures (tax slabs, deposit-insurance cover, rates) are marked unverified and hidden until checked; Hindi and Telugu need native-speaker review.